# Homework Two

**Backward Propagation, Hare and Lynx**

## 1 Theory: Backward Propagation

In this problem, you will derive the equations needed to calculate the gradient of the cost function in a feedforward neural network using back propagation. This gradient is used within learning algorithms (such as gradient descent) that train the neural network.

Recall the output from the $j$th neuron in layer $l$ is given by

$$a_j^{(l)} = g_l(z_j^l) \tag{1}$$

where $g_l$ is a non-linear activation function and

$$z_j^{(l)} = \sum_{i=1}^{n_{l-1}} a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)} \tag{2}$$

where $W_{ij}^{(l)}$ represents the weights and $b_j^{(l)}$ the biases of the network. Assume that the cost function $C$ can be expressed as a sum on the data set $\mathcal{D}$ as

$$C = \frac{1}{\mathcal{D}} \sum_{x \in \mathcal{D}} c(x) \tag{3}$$

and define the notation;

$$\delta_j^{(l)} = \frac{\partial c}{\partial z_j^{(l)}}. \tag{4}$$

a) Show that the quantity $\delta_j^{(l)}$ can be expressed as (the last layer is computed first and it goes backward, hence backward propagation)

$$\delta_j^{(L)} = \frac{\partial c}{\partial a_j^{(L)}} g_L'(z_j^{(L)}) \tag{5}$$
$$\delta_j^{(l)} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} (W_{kj}^{(l+1)})^T g_l'(z_j^{(l)})( \text{ for } l < L)$$

b) Show that the partial derivatives of the cost function with respect to the network's weights and biases can be calculated as

$$\frac{\partial c}{\partial W_{ij}^{(l)}} = a_i^{l-1} \delta_j^{(l)} \tag{6}$$
$$\frac{\partial c}{\partial b_j^{(l)}} = \delta_j^{(l)}.$$

## 2 Numerical: Hare and Lynx

We consider the Lotka-Volterra predator-prey model:

$$\frac{dx}{dt} = \alpha x - \beta xy \tag{7}$$
$$\frac{dy}{dt} = \delta xy - \gamma y$$

where in this model $x$ is the population of hare (in thousands), $y$ is the population of the lynx (in thousands). $\alpha$ is the natural growth rate of the hares when lynxs are absent. $\gamma$ is the death rate of lynx when hares are absent. $\beta$ is the predation rate: at which rate the lynxes kill hare per encounter and $\delta$ is describes how efficient lynxes convert the energy from consuming hare into producing new lynxes. Needly to say, it makes sense for these parameters to be positive.

## 2.1 Synthetic Data

a) Generate synthetic data using

$$\alpha = 0.6, \beta = 0.025, \delta = 0.0125, \gamma = 0.5 \tag{8}$$

with initial conditions

$$x(0) = 30, y(0) = 4 \tag{9}$$

Simulate the system for $t = 0, 1, \ldots, 90$ and plot $x(t)$ and $y(t)$.

b) Approximate time derivatives using central differences:

$$\dot{x}(t_i) \approx \frac{x_{i+1} - x_{i-1}}{2\Delta_t}, \dot{y}(t_i) \approx \frac{y_{i+1} - y_{i-1}}{2\Delta_t} \tag{10}$$

c) Estimate the parameters using linear regression (see the steps in lecture 2 notes.) How are your estimated parameters compared with your real parameters? Simulate again with your estimated numbers and plot it together with your ground truth.

d) You may have found a big discrepancy, as calculating derivatives with central differences requires a lot more data points than every year. Try to generate more data every year. At which point does linear regression give you great results? Plot your prediction with your estimated numbers and your ground truth as you populate the data.

## 2.2 Hudson's Bay Lynx-Hare Data

a) Load the historical lynx-hare dataset (there are two datasets) and plot the time series.

b) Approximate time derivatives using central differences.

c) Construct the regression matrices as before and estimate $\alpha, \beta, \delta, \gamma$ using linear regression.

d) Simulate the Lotka-Volterra system using the estimated parameters and compare with the historical data (for both datasets). (I am not saying this will always work, but plot it anyway).

## 2.3 NN and PINN

a) First train a neural network to fit your synthetic data and the historic data (two sets). Make sure only use part of the data to train and use the rest to predict. How is your prediction compared to the ground truth? What is your expectation?

b) Train a physics-informed neural network as in tutorial 3. You need to submit codes and plots that demonstrates your PINN can predict parameters from the synthetic data even with small (but visible) amount of noise. Apply your PINN to historic data, report your best prediction.

   Here are some tips.

   - The biggest difference from tutorial 3 is that now we need two outputs from the neural network.
   - Make sure your physics loss is computed correctly. We need both first derivatives and no second derivatives. You might want to change the range of time, hare and fox so that the range is roughly 0 to 1, but that will change your differential equations. The pytorch tensor can do some unexpected broadcasting so doubt check your physics loss is computed correctly! I know I am repeating myself, but I also lost hours on this.
   - It is much harder to train over say six period of data. Try to feed just enough data (which means, unlike linear regression, yearly data will do fine. ) to train the network and run long enough to see if the PINN can predict the next period.
   - When you have a working PINN, add some small noise to your synthetic data to see how robust your PINN is.
   - When you feel like the challenge, try to fit the historical data. If your model can handle 10% noise, it should fit `lotvol.csv` reasonably well. The `Leigh1968_harelynx.csv` however, is a huge mess. Try your best to fit it. The historic data is so noisy that it is very hard to tell one fit is better than the other. One hint is that you can probably tell the period by eye. And the period happens to be $T = \frac{2\pi}{\sqrt{\alpha\gamma}}$. There are some biological fun fact that will help you train the parameters: $\gamma$ is between 0.5 and 1, about this fraction of lynx will naturally die without the hares.

Predation rate $\beta$ is between $0.02 - 0.1$.

Good luck and have fun.

If your model works on synthetic data without noise, you already reproduced the result in this paper section 5.4 from November 2023 (just over 2 years ago). And this paper studied the small noisy case(still very far away from the big noisy historical data) with more complicated model in November 2025 (just a few months ago).